Word Level Plagiarism Checker

Multi-document word-level plagiarism detection and similarity analysis using Data Structures and Algorithms



Data Structures and Algorithms-3 (25CS2103E) · A.Y. 2026-2027
Team 16 · Section 13 · Guide: Dr. Dyuti Banerjee
Varshini · Saranya · Manya

Why manual checking falls short

PROBLEM STATEMENT

Manual plagiarism detection is time-consuming and inefficient, making it hard to find matching text across many research papers accurately.

Slow at scale

Comparing many long papers by hand simply doesn't scale.

Closed commercial tools

Turnitin and iThenticate are paid; their internals can't be studied.

A learning gap

A DSA-based checker shows how efficient matching really works.

Objectives

1. Detect word-level plagiarism across multiple documents automatically
2. Use KMP, Rabin–Karp, Z algorithm and hashing for efficient text matching
3. Calculate similarity percentages and highlight matching words and phrases
4. Compare uploaded documents with each other and with a paper repository
5. Generate similarity reports that support review and academic integrity

Existing tools we studied

Turnitin

Compares papers with web pages, publications and past submissions; shows a similarity %.

iThenticate

Checks manuscripts against scholarly sources before publication. Closest to our focus.

Grammarly

Plagiarism checker combined with writing help and citation suggestions.

Copyleaks

Detects identical and paraphrased text; offers an API and source links.

Quetext

Contextual analysis and fuzzy matching with highlighted matches.

Plagiarism Checker X

Bulk cross-comparison of many documents with side-by-side, colour-coded results.

Common thread: a similarity score supports review; it never proves plagiarism on its own.

How the system works



Upload

PDF, DOCX or TXT files

Preprocess

Extract text, tokenize and split sentences

Match

KMP, Rabin–Karp, Z, hashing, suffix arrays

Score

Jaccard, TF-IDF cosine and matched-text %

Report

Highlights, ranked pairs and warning thresholds

Privacy-first: in a private scan, documents stay in memory for the session only. Unpublished work is not added to the permanent repository by default.

Algorithms and data structures

Exact and shared-passage search

KMP · Rabin–Karp (double hash) · Z-function · Aho-Corasick · Suffix array + Kasai LCP

Reworded-text detection

Levenshtein edit distance · Smith-Waterman local alignment · Longest Common Subsequence

Similarity scoring

Jaccard · TF-IDF cosine · shared-passage coverage % · MinHash for fast estimates

Graphs and data structures

Edmonds-Karp sentence pairing · vertex-cover 2-approximation · hash maps · inverted index

Implementation: four core modules

1 Text preparation and hashing

`tokenize()` · `sentences()` · `_enc()`

Splits text into words and sentences; hashes tokens with a polynomial hash (base 131, mod $2^{61}-1$).

2 Longest shared passages

`suffix_array()` · `kasai_lcp()`

Prefix-doubling suffix array in $O(n \log^2 n)$ plus Kasai's linear-time LCP array.

3 Paraphrase-tolerant alignment

`smith_waterman()`

Local alignment (match +2, mismatch −1, gap −1) finds similar regions despite small edits.

4 One-to-one sentence pairing

`edmonds_karp()`

BFS-based max-flow on a bipartite graph pairs each sentence with its best match.

Output: SimilarityLab dashboard



Home screen: private scan on, 25% matched-text threshold, five uploaded documents



Pair cards: the unpublished manuscript vs paper C is flagged at 39.4% matched text

Output: scores and phrase search

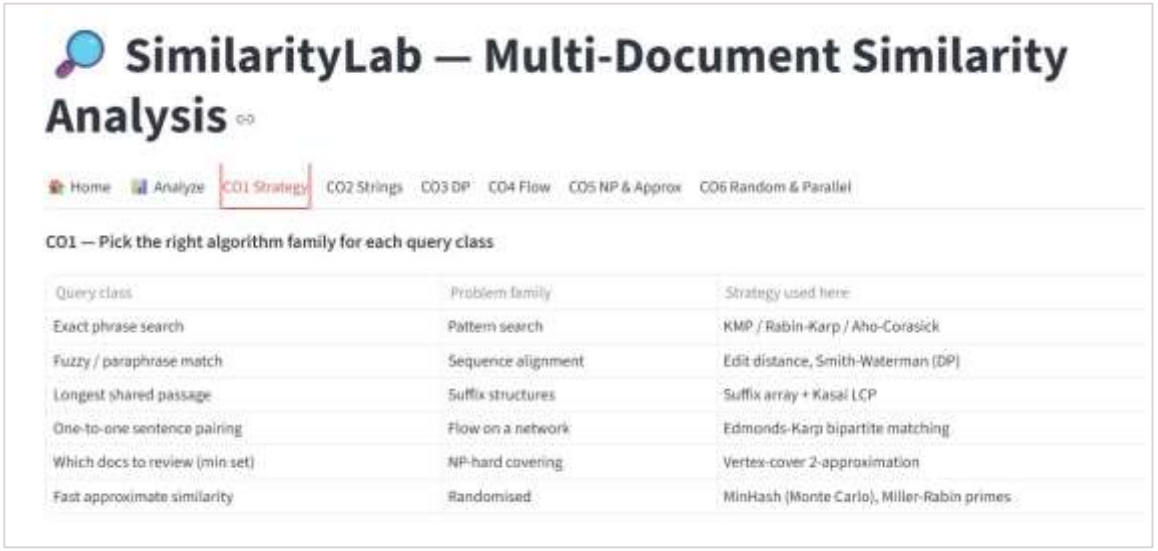


Full score table: Jaccard, TF-IDF cosine and shared-passage coverage for every pair



Phrase search with KMP, Rabin–Karp, Z-function and Aho-Corasick across all documents

Output: choosing the right algorithm



SimilarityLab — Multi-Document Similarity Analysis

Home | Analyze | CO1 Strategy | CO2 Strings | CO3 DP | CO4 Flow | CO5 NP & Approx | CO6 Random & Parallel

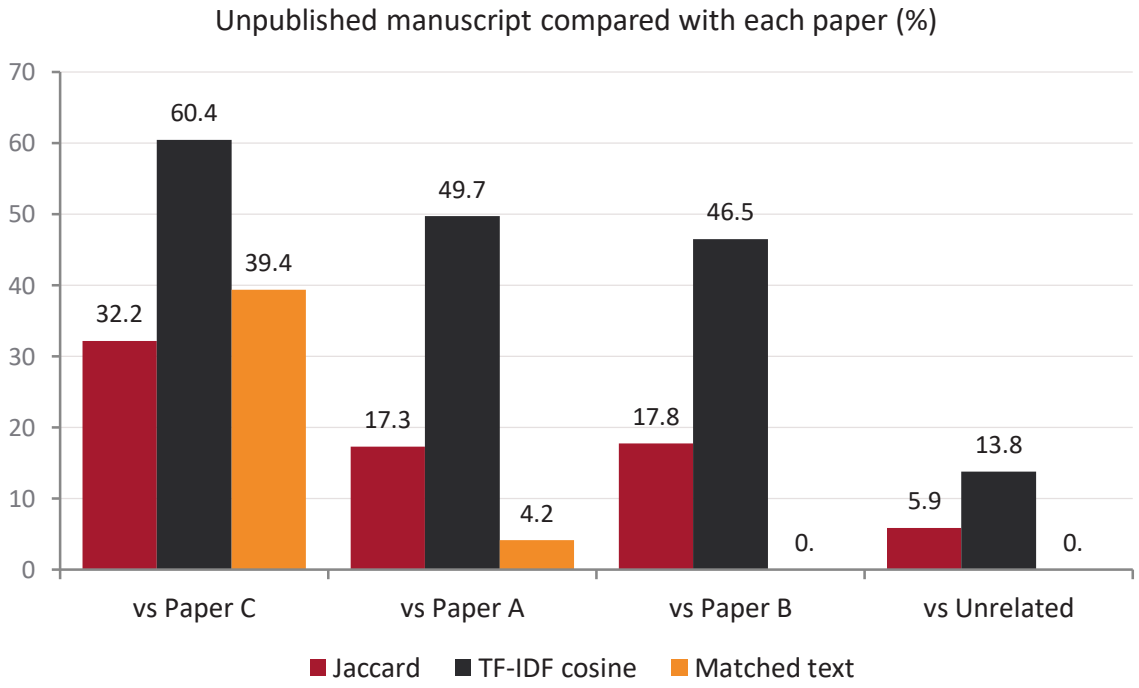
CO1 — Pick the right algorithm family for each query class

Query class	Problem family	Strategy used here
Exact phrase search	Pattern search	KMP / Rabin-Karp / Aho-Corasick
Fuzzy / paraphrase match	Sequence alignment	Edit distance, Smith-Waterman (DP)
Longest shared passage	Suffix structures	Suffix array + Kasai LCP
One-to-one sentence pairing	Flow on a network	Edmonds-Karp bipartite matching
Which docs to review (min set)	NP-hard covering	Vertex-cover 2-approximation
Fast approximate similarity	Randomised	MinHash (Monte Carlo), Miller-Rabin primes

Each query class maps to an algorithm family

- Exact phrase: KMP, Rabin–Karp, Aho-Corasick
- Reworded text: Smith-Waterman
- Longest passage: suffix array + LCP

Results on the demo set



39.4%

matched text between the unpublished manuscript and repository paper C

High similarity: manual review recommended (threshold 25%)

What the results tell us

What worked

- The true source ranked highest, with 39.4% matched text
- Matched-text % clearly separated copied, minimal and unrelated pairs
- Jaccard and cosine stay high for same-topic papers, so don't rely on them alone

Limits to keep in mind

- Demo papers were written by us with known overlap
- Not a test on a large real-world collection
- Only the uploaded documents are compared
- Heavy paraphrase or translation can lower scores
- Every flagged pair needs human review

Future scope

Short-term

- Better preprocessing and more file formats
- Higher matching accuracy and clearer highlighting
- Improved reports and scan history

Medium-term

- Larger open-access paper repositories
- Semantic similarity for paraphrased content
- Faster processing of large collections

Long-term

- Machine learning and NLP analysis
- Web platform with login and secure storage
- Multilingual detection and citation checks

Conclusion

- Applied KMP, Rabin–Karp, suffix arrays, Smith-Waterman and max-flow to a real problem
- Matched-text % separates true copying from topical overlap
- Privacy-first by design; scores guide review, people decide

Thank you

Questions?

Varshini · 2520080013
Saranya · 2520080012
Manya · 2520090032